장대산

Java / Spring
Backend Developer

가족별로 달라지는 권한을 데이터 구조에 반영하고,
배포를 취소해도 운영 중인 서비스가 유지되도록 구현했습니다.

Familink 백엔드 및 인프라

가족 구성원의 관계와 활동 기록을 관리하고,
사진·영상 업로드 기능을 개발했습니다.

Familink 프로젝트 보기

잘타슈 인프라

자전거 현황과 예측 정보를 제공하는 서비스의
실행 환경을 연결하고, 배포와 복구 과정을 개선했습니다.

잘타슈 프로젝트 보기

GitHub · daesan12 Portfolio · daesan12.github.io Email · diqkraud@naver.com

familink

떨어져 지내는 가족도 서로의 하루를 알 수 있도록

사진과 글로 일상을 공유하고,
어르신과의 AI 안부 통화 내용을
리포트로 확인하는 서비스입니다.

백엔드 및 인프라

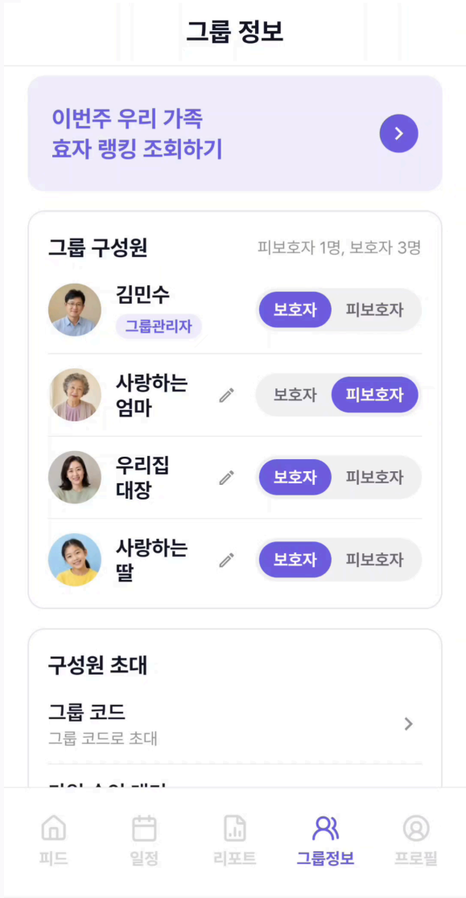
그룹·피드 API와 접근 권한

가족별 역할과 일일 적립 규칙

S3 업로드와 AWS 배포 환경

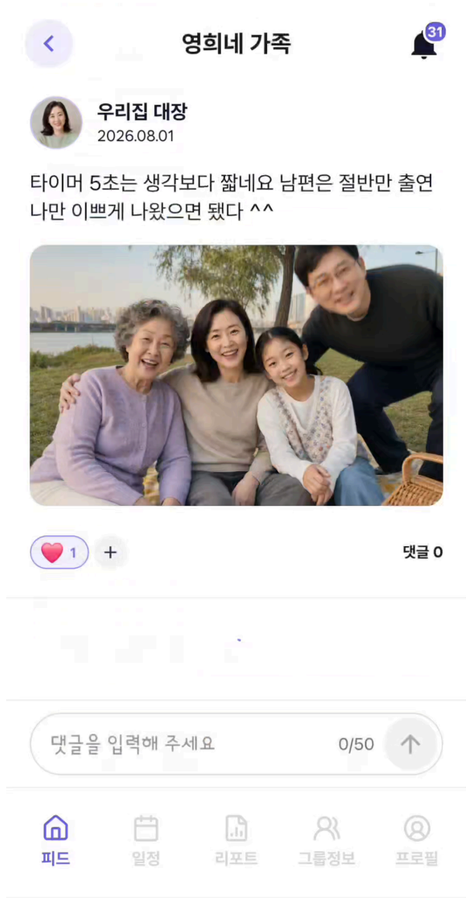
6인 팀 · 2026.07.13–08.10

GitHub · Familink



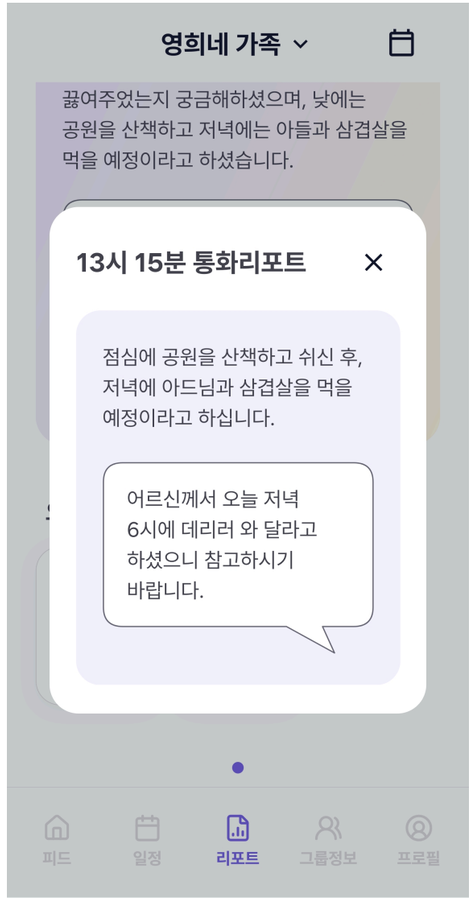
가족 그룹

가족 초대와 구성원 역할 관리



가족 피드

사진·글, 댓글과 반응



안부 리포트

AI 통화 리포트 · 팀 서비스 기능

팀 서비스 시연 화면

같은 사람도 가족마다 역할이 다를 수 있습니다

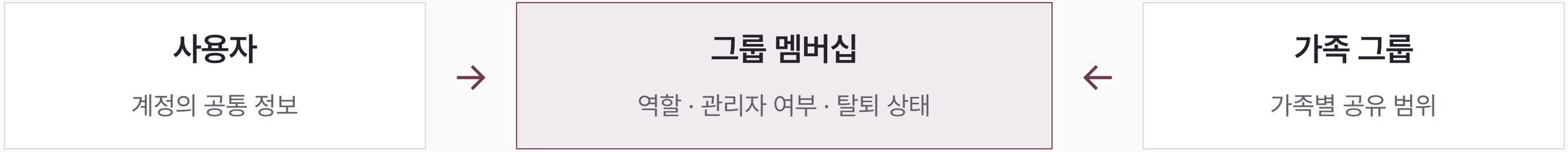
한 사람이 여러 가족에 참여한다면, 계정 하나에 역할을 하나만 저장해서는 관계를 표현하기 어렵습니다.

계정에 가족과 역할을 저장

구조는 단순하지만, 여러 가족에 소속되거나 가족마다 역할이 달라질 때 표현에 제약이 생깁니다.

가족과의 관계를 별도로 저장

계정에는 공통 정보를 두고, 구성원 정보에서 가족별 역할과 활동 상태를 관리합니다.



접근 권한은 API에서 확인

활성 구성원인지 검사하고, 수정 등 작업별로 작성자 본인인지 확인합니다.

중복 관리자 상태는 DB에서 제한

조건부 UNIQUE 제약으로 그룹별 활성 관리자를 최대 한 명으로 제한했습니다.

최대 한 명을 제한하며, 반드시 한 명의 존재를 보장하는 제약은 아닙니다.

게시글을 삭제해도, 그날의 적립은 사라지지 않도록

보호자가 글을 여러 번 작성하거나 삭제 후 다시 써도, 카네이션은 하루 한 번만 적립돼야 합니다.

현재 게시글만으로 판단한다면

글을 삭제한 뒤에는 이미 적립받았는지 알기 어렵습니다. 게시글과 적립 기록을 분리해, 글이 없어져도 그날의 적립 이력은 남겼습니다.

> **멤버십 + KST 날짜**를 UNIQUE로 묶고,
> 중복 요청은 ON CONFLICT로 무시합니다.

먼저 조회한 뒤 저장하는 방식에서 생길 수 있는 동시 요청의 중복을 DB 규칙으로 막았습니다.

동시 요청에도 적립 기록은 한 건

프로젝트 이후 격리 DB 시험에서
독립 트랜잭션 6개를 동시에 실행했습니다.

1 저장

5 중복 무시

3회 반복 · 매번 최종 기록 1건

PostgreSQL 저장 규칙 검증

글 작성 · 적립 → 글 삭제 · 적립 기록 유지 → 같은 날 재작성 · 중복 적립 없음

서버에서는 됐는데, 앱에서는 사진이 올라가지 않았습니다

가족 피드의 사진·영상은 팀의 API 계약에 따라 앱에서 S3로 직접 전송했습니다.

> **서버 중계** 파일 처리를 서버에서 관리하기 쉽지만 파일 본문도 백엔드를 통과합니다.
>
> **앱 직접 전송** 백엔드의 파일 중계는 줄지만 앱의 요청과 서명 조건을 맞춰야 합니다.

주소 발급 / curl 파일 전송

서버·테스트 도구에서는 정상

→

실제 모바일 앱

같은 업로드 흐름에서 실패

실제 파일과 요청 조건을 비교

앱이 읽은 파일, 주소 발급에 사용한 정보와 실제 전송 헤더를 대조했습니다. 실제 Blob으로 주소를 요청하고 같은 Blob을 전송하도록 맞췄습니다.

서명 조건과 저장 확인

Content-Length를 서명에서 제외하고 Content-Type은 유지했습니다. 주소 발급과 함께 HeadObject로 저장된 파일의 존재·크기·형식을 확인하는 기능을 구현했습니다.

후속 로컬 서명 조건 테스트 3건 통과

URL 발급 Provider 회귀 테스트

Test	Duration	Result
keepsObjectKeyUnderPurposeNamespace()	0.006s	passed
returnsOnlyContentTypeHeader()	0.278s	passed
signsContentTypeWithoutContentLength()	0.005s	passed

잘타슈

지금 자전거가 있어도,
도착할 때는 없을 수 있으니까

이동하는 동안 자전거 수는 달라집니다. 현재 현황과 도착 시점의 예상 잔여 수를 확인하고 경로를 비교하는 서비스입니다.

인프라

팀 모듈의 AWS 실행·연결 환경

Blue/Green 배포와 Jenkins 복구

MySQL 백업과 격리 복원 검증

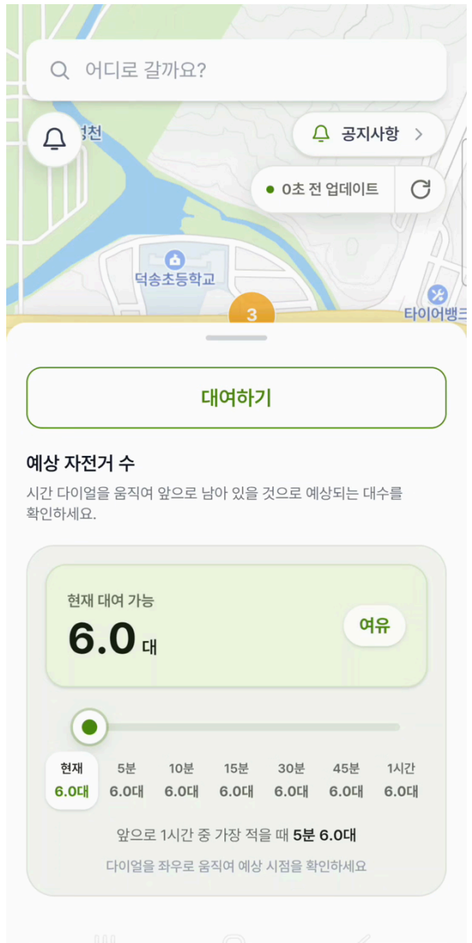
6인 팀 · 2026.08.18–09.28
삼성청년SW·AI아카데미
우수 프로젝트 선정 · 팀 수상

GitHub · JalTaShu



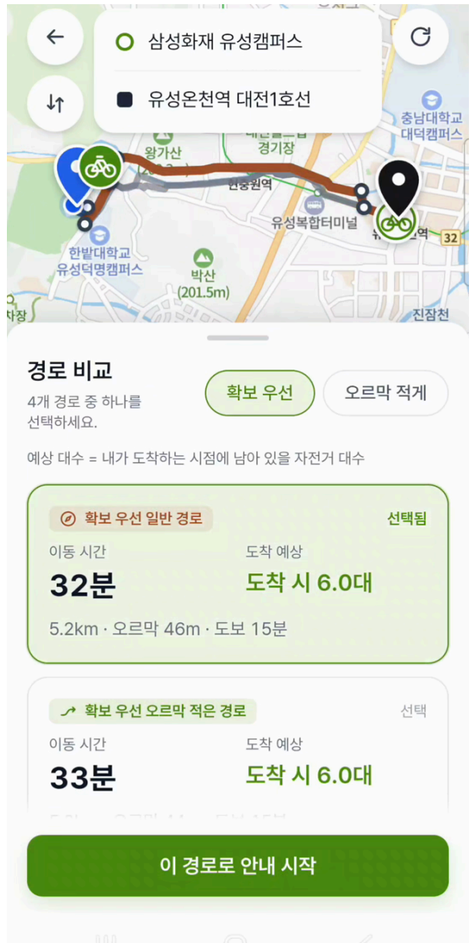
대여소 탐색

지도에서 현재 현황 확인



도착 시점 예측

이동 시간에 맞춘 예상 수량



경로 안내

대여소까지 이동 경로 비교

팀 서비스 시연 화면

사용자 요청과 데이터 처리 작업을 어떻게 나눌까?

사용자가 정보를 조회하는 동안에도 현황 수집, 과거 데이터 처리와 예측 결과 생성이 이어져야 합니다.

한 서버에 모두 실행

연결과 관리가 단순해지는 대신, 요청 처리·데이터 처리·AI·빌드가 같은 자원을 사용합니다.

역할에 따라 실행 위치 분리

서로 다른 작업을 나눠 관리할 수 있지만, 서버 간 통신과 저장소 접근 권한을 맞춰야 합니다.

EC2 A · 사용자 요청과 데이터 저장

Nginx, Frontend/Backend, Collector
MySQL, Redis, OSRM

EC2 B · 데이터 처리와 AI 실행

Kafka, Spark, AI Serving
Jenkins, 모니터링

현재	Collector → Redis DB0 → Backend
이력	Collector → Kafka → Spark → MySQL
예측	MySQL → AI Serving → Redis DB2 → Backend

직접 맡은 실행·연결 범위

팀원이 개발한 모듈의 Docker 실행 환경, 내부 연결과 접근 권한을 구성했습니다.
Redis 사용 영역을 담당자들과 구분하고 AI 환경의 DB2 연결을 확인했습니다.

두 EC2는 역할 분리이며 이중화는 아닙니다. DB0/DB2는 같은 Redis의 논리적 구분입니다.

새 버전을 확인하기 전까지 기존 서비스는 유지해야 했습니다

새 버전의 문제가 기존 서비스에 영향을 주지 않도록, 별도로 실행해 확인한 뒤 수동 승인으로 전환했습니다.
이 과정에서 승인 취소 후 후보 컨테이너와 상태 파일이 남는 문제를 발견했습니다.

남겨야 할 것

현재 요청을 처리하는 active 슬롯
다른 배포가 만든 후보

정리할 것

이번 취소 요청의 후보 컨테이너
해당 후보의 상태 파일

삭제 전에 배포와 상태를 비교

요청 SHA와 후보 SHA가 일치하는지 확인하고, active 슬롯 삭제는 거부하도록 했습니다.

후처리 실패까지 수정

SSH 계정 불일치로 정리에 실패한 문제를 고친 뒤 승인 취소를 다시 시험했습니다.

후보 제거 → **active 유지** → **서비스 정상 응답**

동일 서버의 Blue/Green 슬롯은 운영 DB를 공유합니다. 이미지 복구와 DB 변경 복구는 별개입니다.

한 번 등록한 설정이 재시작 후에도 남는 것은 아니었습니다

Job과 빌드 기록은 남아 있었지만, GitLab 인증 정보가 사라져 자동 빌드가 실패했습니다.

화면 등록과 시작 설정의 차이

UI에서 등록한 인증 정보가 JCasC에는 없었습니다. 재시작하며 설정 파일을 다시 적용하는 과정에서 제거됐습니다.

JCasC	인증 정보의 정의
Docker Secret	실제 비밀값

화면에서 다시 등록하면 당장은 동작해도, 설정을 재적용할 때 반복될 수 있습니다. 시작 설정에 정의해 재시작할 때도 적용되도록 바꿨습니다.

실제 재시작 후 확인

- GitLab 인증 정보 유지
- Job·빌드 기록·Agent 연결 유지
- Polling 자동 빌드 성공

```
[Pipeline] }
[Pipeline] // withEnv
[Pipeline] }
[Pipeline] // node
[Pipeline] End of Pipeline
Finished: SUCCESS
```

자동 빌드 완료 로그

백업 파일을 저장하는 것과 복원할 수 있다는 것은 다릅니다

백업 파일이 존재하는지만으로는 다시 읽을 수 있는지 알 수 없습니다. 저장과 복원 확인을 나눴습니다.

백업 생성 → S3 보관·재다운로드 → 격리 MySQL 복원

파일 보관에서 복원 확인까지

중복 실행을 막고 임시 파일에 백업했습니다. gzip 검사 후 이름을 확정하고, checksum을 만든 뒤 S3에 업로드했습니다.

백업을 다시 내려받아 격리 MySQL에서 읽었습니다. 운영 DB에 영향을 주지 않고 SQL 실행과 스키마 생성을 확인할 수 있는 방식입니다.

확인한 결과

파일	재다운로드한 파일의 checksum 일치
복원	격리 MySQL에서 SQL import 완료
구조	운영·스테이징 스키마 존재 확인

파일 일치와 SQL import·스키마 복원 범위를 확인했습니다.

장대산 Java / Spring Backend Developer

GitHub · daesan12 Portfolio · daesan12.github.io Email · diqkraud@naver.com